

Interfete grafice in Java

Abstract Windowing Toolkit

Abstract Windowing Toolkit, pe scurt AWT, este un set de clase cu ajutorul cărora puteți crea o interfață grafică utilizator care să reacționeze la datele de intrare primite de la mouse și tastatură.

Folosind AWT, o interfață este compusă din următoarele:

Componente. Orice poate fi plasat pe o interfață utilizator, cum ar fi butoane, liste derulante, meniuri pop-up, casete de validare sau câmpuri de text.

Containere. Acestea sunt componente care pot conține alte componente. Ați folosit deja un astfel de container - fereastra Applet; alte exemple ar fi panouri, casete de dialog sau ferestre independente.

Administratori de dispunere. Obiecte care definesc modul cum sunt aranjate (dispuse) componentele într-un container.

```
import java.awt.*;
```

Componentele interfetei utilizator

Adăugarea componentelor într-un container

O componentă este adăugată într-un container astfel:

Creați componenta.

Apelați metoda **add()** a containerului, pentru componenta respectivă.

Etichete

Pentru a crea o etichetă folosiți unul dintre următorii constructori:

Label() creează o etichetă goală (vidă), cu textul aliniat la stânga.

Label (*String*) creează o etichetă cu șirul de text dat, aliniat, de asemenea, la stânga.

Label (*String, int*) creează o etichetă cu șirul de text dat și alinierea indicată de argumentul întreg. Pentru stabilirea alinierii se folosesc următoarele variabile de clasă: Label.RIGHT, Label.LEFT, Label.CENTER.

Butoane

Butoanele (zone pe care se poate efectua clic) pot fi create folosind clasa `Button`. Butoanele sunt folosite într-o interfață pentru a declanșa o acțiune, cum este butonul `Quit` (Terminare) folosit pentru a părăsi un program.

Pentru a crea un buton, folosiți unul dintre constructorii:

`Button()` creează un buton care nu conține nici un text pentru explicarea funcției sale.

`Button(String)` creează un buton pe care este afișat șirul de text primit ca argument.

După crearea unui buton, puteți să îi modificați eticheta folosind metoda **`setLabel (String)`** sau puteți afla textul scris folosind metoda **`getLabel()`**

Casete de validare

Casetele de validare (check boxes) sunt mici casete, etichetate sau nu, care pot fi validate („bifate”) sau goale.

În mod normal, casetele de validare sunt **neexclusive**

Aceste componente pot fi organizate și în grupuri de validare, care mai sunt denumite și **butoane radio** (radio buttons).

Checkbox() creează o casetă de validare neetichetată, care nu este validată.

Checkbox(String) creează o casetă de validare nevalidată și care are ca etichetă șirul dat.

Metoda **setState** (boolean) modifică starea acestuia

Metoda **getState()** returnează o valoare Boolean care indică starea de validare a casetei.

CheckboxGroup

```
CheckboxGroup radio = new CheckboxGroup();
```

Obiectul **CheckboxGroup** păstrează starea tuturor casetelor de validare din grupul său. Acest obiect va fi folosit ca argument suplimentar pentru constructorul **Checkbox**.

Checkbox (*String, GrupCaseteValidare, boolean*)

Liste de opțiuni

Listele de opțiuni (choice lists), create din clasa **Choice**, sunt componente care permit alegerea unei sin

```
Choice sex = new Choice();
```

Elementele se adaugă într-o listă de opțiuni folosind metoda `addItem(String)` a obiectului. Următoarele instrucțiuni adaugă două elemente în lista de opțiuni sex:

```
sex.addItem("Masculin");
```

```
sex.addItem("Feminin");
```

Puteți continua să folosiți metoda `addItem()` pentru a adăuga opțiuni în listă chiar și după ce lista a fost introdusă într-un container.

gure opțiuni dintr-o listă derulantă (pull-down list).

Câmpuri de text

Câmpurile de text (text fields) sunt folosite pentru a crea componente în care textul poate fi modificat de utilizator. Aceste componente sunt create din clasa **TextField**.

Pentru a crea un câmp de text, folosiți unul din următorii constructori:

TextField() creează un câmp gol, fără o lățime specificată.

TextField(int) creează un câmp gol, care are o lățime suficientă pentru a afișa numărul specificat de caractere.

TextField(String) creează un câmp completat cu șirul dat și fără o lățime specificată.

TextField (String, int) creează câmp completat cu șirul dat și cu lățimea specificata de argumentul întreg.

```
TextField nume = new TextField(30) ;
```

```
TextField nume = new TextField("Ion I. Ionescu", 30);
```

Zone de text

Zonele de text (text areas), create din clasa **TextArea**, sunt câmpuri de text modificabile care pot conține mai multe linii de text. Zonele de text posedă bare de defilare orizontale și verticale, care permit utilizatorului să parcurgă întreg textul conținut în componentă.

Pentru a crea o zonă de text puteți folosi unul din următorii constructori:

TextArea() creează o zonă de text goală, cu înălțime și lățime nespecificate.

TextArea(int, int) creează o zonă goală, care conține numărul de rânduri dat de primul argument și are lățimea în caractere dată de al doilea argument.

TextArea(String) creează o zonă de text care conține șirul specificat și are lățimea și înălțimea nespecificate.

TextArea(String, int, int) creează o zonă de text care conține șirul specificat, numărul de rânduri fiind dat de primul argument, iar lățimea în caractere de al doilea argument.

Liste de derulare

Listele de derulare (scrolling lists), create din clasa **List**

Lista de derulare poate fi configurată așa încât să poată fi selectate mai multe opțiuni la un moment dat.

Listele de derulare se prezintă asemănător unei zone de text în care sunt afișate mai multe opțiuni

List() creează o listă de parcurgere vidă care permite selectarea unui singur element la un moment dat.

List(int, boolean) creează o listă de derulare care posedă numărul de elemente vizibile indicat de primul argument

După crearea unui obiect List se folosește metoda **addItem(*String*)**

Bare de derulare și glisoare

Barele de derulare (scrollbars) sunt componente care permit selectarea unei valori prin deplasarea unei casete între două săgeți. O bară de derulare poate fi orizontală sau verticală.

Pentru a crea o bară de derulare puteți folosi următorii constructori:

Scrollbar() creează o bară de derulare verticală care are valorile maximă și minimă setate inițial la 0.

Scrollbar(int) creează o bară de derulare care are valorile maximă și minimă setate la 0, iar orientarea este dată de valoarea întreagă. Argumentul poate lua următoarele valori, care sunt variabile de clasă: **Scrollbar.HORIZONTAL** și **Scrollbar.VERTICAL**.

Puteți folosi, de asemenea, un al treilea constructor, care primește cinci argumente:

Scrollbar (int, int, int, int, int). Argumentele acestei metode sunt, în ordine, următoarele:

Orientarea, care este Scrollbar.HORIZONTAL sau Scrollbar.VERTICAL.

Valoarea inițială a barei de derulare, care trebuie să se afle între valorile minimă și maximă ale barei sau să fie egală cu una dintre acestea.

Lățimea sau înălțimea generală a casetei folosite pentru modificarea valorii barei de derulare. Aceasta poate fi egală cu 0 atunci când se folosește dimensiunea prestabilită.

Valoarea minimă a barei de derulare.

Valoarea maximă a barei de derulare.

Suprafețe de desenare

Suprafețele de desenare (canvas) sunt componente folosite, în principal, drept loc de afișare pentru imagini sau animație. Puteți desena și pe alte componente, însă obiectele Canvas sunt cele mai simplu de folosit în acest scop.

Pentru a folosi o suprafață de desenare trebuie să creați o subclasă a clasei Canvas. Această subclasă se poate ocupa de toate operațiunile de desenare care trebuie să aibă loc, în metoda sa `paint()`.

O dată creată o subclasă Canvas, aceasta poate fi folosită în program prin apelarea constructorului său și prin adăugarea noului obiect Canvas într-un container.

Aranjarea componentelor într-o interfață utilizator

Disponerea componentelor interfeței

Un administrator de dispunere determină modul cum vor fi aranjate (dispuse) componentele adăugate într-un container.

Administratorul de dispunere implicit este clasa **FlowLayout**. Această componentă permite dispunerea secvențială a obiectelor, de la stânga la dreapta, în ordinea în care acestea sunt adăugate în container. Atunci când nu mai este loc pe un rând, celelalte obiecte se dispun în continuare pe rândul următor, continuând secvența de la stânga la dreapta.

Biblioteca AWT conține cinci administratori de dispunere principali: **FlowLayout**, **GridLayout**, **BorderLayout**, **CardLayout** și **GridBagLayout**. Pentru a crea un administrator de dispunere pentru un container, se creează o instanță a clasei container folosind o instrucțiune de genul:

```
FlowLayout flo = new FlowLayout();
```

Dispunerea secvențială

FlowLayout este cea mai simplă clasă de administrare a dispunerii. Ea aranjează componentele într-o manieră asemănătoare dispunerii cuvintelor într-o pagină - de la stânga la dreapta până la capătul rândului, apoi în continuare pe rândul următor.

În mod prestabilit, dacă folosiți constructorul FlowLayout() fără argumente, componentele de pe fiecare rând vor fi centrate. Dacă doriți ca acestea să fie aliniate la marginea din stânga sau din dreapta a containerului, trebuie folosite drept argument pentru constructor variabilele de clasă FlowLayout.LEFT sau FlowLayout.RIGHT:

```
FlowLayout dreapta = new FlowLayout(FlowLayout.RIGHT);
```

Variabila de clasă FlowLayout.CENTER este folosită pentru dispunerea pe centru a componentelor.

Dispunerea tabelară

Administratori de dispunere tabelară (`grid`) aranjează componentele într-un caroi (tabel) format din rânduri și coloane.

Componentele sunt adăugate începând cu celula aflată cel mai în stânga pe primul rând al tabelului și continuând spre dreapta. După completarea tuturor celulelor de pe primul rând se continuă cu cel de-al doilea rând, de la stânga la dreapta, și așa mai departe.

Dispunerea tabelară este stabilită pornind de la clasa **GridLayout**. Constructorul **GridLayout** primește două argumente - numărul de rânduri și numărul de coloane din tabel. Următoarea instrucțiune creează un administrator de dispunere tabelară pentru 10 rânduri și 3 coloane:

```
GridLayout gr = new GridLayout(10,3);
```

Ca și în cazul dispunerii secvențiale, dacă se folosesc două argumente suplimentare, puteți specifica spațiul pe orizontală și pe verticală care trebuie lăsat între componente. Următoarea instrucțiune creează un tabel cu 10 rânduri, 3 coloane, un spațiu pe orizontală de 5 pixeli și un spațiu pe verticală de 8 pixeli:

```
GridLayout gr2 = new GridLayout(10, 3, 5, 8);
```

Spațiul între componentele dispunerii tabelare este implicit 0 pixeli, atât pe orizontală cât și pe verticală.

Dispunerea marginală

Dispunerile marginale (border layouts), create pornind de la clasa **BorderLayout**, împart un container în cinci secțiuni: nord, sud, est, vest și centru.

Folosind dispunerea marginală, componentele din cele patru puncte cardinale vor ocupa spațiul de care au nevoie, iar centrul obține spațiul rămas disponibil. De obicei, acest aranjament are ca rezultat o componentă centrală de dimensiuni mari, înconjurată de patru componente mai „subțiri”.

Dispunerea marginală se obține folosind unul dintre constructorii **Borderlayout()** sau **BorderLayout(int, int)**. Primul constructor creează o dispunere marginală fără nici un spațiu între componente. Al doilea constructor specifică spațiul pe orizontală, respectiv pe verticală.

După ce ați creat o dispunere marginală și ați stabilit-o drept administrator curent de dispunere pentru un container, componentele sunt adăugate folosind o altă apelare a metodei **add()** decât cele folosite anterior:

add(String, *componenta*)

Al doilea argument al acestei metode este componenta care trebuie adăugată în container. Primul argument este un șir care indică zona unde va fi plasată componenta. Există cinci valori posibile: "North" (nord), "South" (sud), "East" (est), "West" (vest) și "Center" (Centru).

Următoarea instrucțiune adaugă un buton, denumit butonTenninare, în porțiunea nordică: `add( "North" , butonTerminare) ;`

Combinarea administratorilor de dispunere

Pentru a găsi dispunerea potrivită, de cele mai multe ori trebuie folosiți mai mulți administratori pentru aceeași interfață. Acest lucru se poate face adăugând mai multe containere într-un container principal, fiecare dispunând de propriul administrator de dispunere.

Aceste containere se numesc panouri (**panels**) și sunt derivate din clasa **Panel**. Panourile sunt containere folosite pentru a grupa diferite componente. Când lucrați cu panouri trebuie să țineți cont de următoarele:

Panoul se umple cu componente înainte de a fi introdus într-un container mai mare;

Panoul posedă propriul administrator de dispunere.

Crearea unui panou:

```
Panel panou=new Panel();
```

Panoului i se atribuie o metodă de dispunere prin apelarea metodei `setLayout()`, ca în exemplul:

```
BorderLayout b1=new BorderLayout();
```

```
panou.setLayout(b1);
```

Componentele se adaugă apoi folosind metoda **add()**.

Tratarea evenimentelor

Tipuri de evenimente

Clicuri cu mouse-ul. Apăsarea butonului mouse-ului (mouse down), eliberarea acestuia (mouse up) sau apăsarea și eliberarea butonului pe aceeași poziție (mouse clic).

Mișcări ale mouse-ului. Cursorul mouse-ului care părăsește sau pătrunde pe suprafața ocupată de o componentă a interfeței sau tragerea cu mouse-ul (drag - mișcările efectuate de cursor atunci când butonul mouse-ului este apăsat).

Apăsări de taste. Apăsarea tastei, eliberarea tastei, tastarea (apăsarea și eliberarea unei taste).

Evenimente ale interfeței utilizator. Execuția de clicuri pe butoane, derularea listelor, afișarea meniurilor derulante și așa mai departe.

Metoda `handleEvent()`

Tratarea evenimentelor reprezintă domeniul în care au apărut cele mai multe modificări în trecerea de la Java 1.02 la Java 2. Evenimentele sunt generate și executate aproximativ în același mod, indiferent de versiunea de limbaj pe care o folosiți; diferența constă în modul cum acestea sunt recepționate și procesate.

În Java 1.02, toate evenimentele care apar în ciclul de viață al unui program sunt tratate de o metodă denumită **`handleEvent()`**. Aceasta este definită în clasa **`Component`**, care este moștenită de `java.applet.Applet`, devenind astfel disponibilă tuturor applet-urilor.

Atunci când un eveniment este transmis metodei **`handleEvent()`**, aceasta apelează apoi o metodă specifică de tratare a evenimentului, în funcție de tipul acestuia. Exemple de astfel de metode specifice sunt `mouseDown()`, `mouseUp()` sau `keyDown()`.

Pentru a trata un eveniment, va trebui să suprascrieți una dintre aceste metode specifice. Apoi, la apariția evenimentului respectiv, metoda respectivă va fi apelată. De exemplu, puteți suprascrie metoda `mouseDown()` pentru a afișa un mesaj. Atunci când apare un eveniment de acest tip (apăsarea butonului mouse-ului), mesajul va fi afișat.

Tratarea clicurilor de mouse

Evenimentele mouse down și mouse up

```
public boolean mouseDown(Event evt, int x, int y) {  
    //...  
}
```

Metoda **mouseDown()** (ca și metoda **mouseUp()**) primește trei parametri: evenimentul în sine și coordonatele x și y unde a apărut evenimentul respectiv.

Argumentul **evt** reprezintă o instanță a clasei Event.

Toate evenimentele generează o instanță a clasei Event, care conține informații privind locul și perioada când a avut loc evenimentul, tipul lui, precum și alte informații.

Tratarea mișcărilor mouse-ului

Evenimentele mouse drag și mouse move

```
public boolean mouseMove(Event evt, int x, int y) {  
    // ...  
}  
  
public boolean mouseDrag(Event evt, int x, int y) {  
    // ...  
}
```

Evenimentele mouse enter și mouse exit

```
public boolean mouseEnter(Event evt, int x, int y) {  
    // ...  
}  
  
public boolean mouseExit(Event evt, int x, int y) {  
    // ...  
}
```

Tratarea evenimentelor de tastatură

Un eveniment de tastatură este generat ori de câte ori utilizatorul apasă o tastă.

Pentru ca evenimentul de tastatură să fie recepționat de o componentă, aceasta trebuie să fie selectată (**focused** - „în centrul atenției”; cu alte cuvinte, să fie acea componentă a interfeței selectată pentru a primi datele de intrare).

Pentru a indica explicit că o componentă este selectată pentru a primi date de intrare (input focus), poate fi apelată metoda **requestFocus()** a componentei, fără argumente.

Evenimentele key down și key up

```
public boolean keyDown(Event evt, int tasta) {  
    // ...  
}
```

Taste prestabilite

Tastele standard definite în clasa Event.

<i>Variabila de clasă</i>	<i>Tasta reprezentată</i>	.
Event.HOME	Tasta Home	
Event.END	Tasta End	
Event.PGUP	Tasta Page Up	
Event.PGDN	Tasta Page Down	
Event.UP	Tasta Up	
Event.DOWN	Tasta Down	
Event.LEFT	TastaLeft	
Event.RIGHT	Tasta Right	
Event.F1	Tasta F1	
Event.F2	Tasta F2	
Event.P3	Tasta F3	
Event.F4	Tasta F4	
Event.F5	TastaF5	
Event.F6	Tasta F6	
Event.F7	Tasta F7	
Event.F8	Tasta F8	
Event.F9	Tasta F9	
Event.F10	Tasta F10	
Event.F11	TastaF11	
Event.F12	TastaF12	

```
if (tasta = Event.UP) {  
    // ...  
}
```

Tratarea evenimentelor componentelor

Următoarele evenimente pot fi generate prin interacțiunea cu componentele interfeței:

Evenimente de acțiune (action events). Sunt principalele evenimente ale majorității componentelor interfeței, prin care se indică faptul că acestea au fost "activate". Evenimentele de acțiune sunt generate atunci când este apăsat un buton, când se selectează sau se deselectează o casetă de validare sau un buton radio, când se alege o opțiune dintr-un meniu sau când utilizatorul apasă tasta Return sau Enter într-un câmp de text.

Evenimente de selectare sau deselectare a listelor (list select/deselect events). Aceste evenimente sunt generate atunci când se selectează o casetă de validare sau o opțiune dintr-un meniu (listă). Aceste exemple declanșează, de asemenea un eveniment de acțiune.

Evenimente de primire sau cedare a selecției (got/lost focus events). Aceste evenimente pot fi generate de orice componentă, fie ca răspuns la un clic de mouse, fie la selecționarea prin tasta Tab. Primirea selecției (got focus) înseamnă că acea componentă este selecționată și poate fi activată sau poate primi date de intrare. Cedarea selecției (lost focus) înseamnă că s-a selecționat o altă componentă.

Tratarea evenimentelor de acțiune

```
public boolean action(Event evt, Object arg) {  
    // ...  
}
```

<i>Componente</i>	<i>Tip de argument</i>	<i>Ce conține</i>
Butoane	String	Eticheta butonului
Casete de validare	Boolean	Întotdeauna true
Butoane radio	Boolean	Întotdeauna true
Meniuri de opțiuni	String	Eticheta elementului selectat
Câmpuri de text	String	Textul din cadrul câmpului

```
public boolean action(Event evt, Object arg) {  
    if (evt.target instanceof Button) {  
        String eticheta = (String)arg;  
        if (eticheta.equals("OK"))  
            // trateaza butonul OK  
        else if (eticheta.equals ("Anulare"))  
            // trateaza butonul Anulare  
        else if (eticheta.equals("Navigare"))  
            // trateaza butonul Navigare  
  
            // ...  
        }  
    }  
}
```

Tratarea evenimentelor de selectare

```
public boolean gotFocus(Event evt, Object arg) {
```

```
// ...
```

```
}
```

```
public boolean lostFocus(Event evt, Object arg) {
```

```
// ...
```

```
}
```

Evenimente ale zonelor de text

Zonele de text prezintă aceleași evenimente ca și câmpurile de text. Puteți folosi metodele `gotFocus()` și `lostFocus()` pentru a intercepta evenimentele de primire/cedare a selecției:

```
public boolean gotFocus(Event evt, Object arg) {  
    // ...  
}  
  
public boolean lostFocus(Event evt, Object arg) {  
    // ...  
}
```

Evenimente ale listelor derulante

Listele derulante generează trei tipuri de evenimente diferite; selectarea sau deselectarea unui element individual al listei are ca efect un eveniment de selectare/deselectare listă, iar dublul clic executat pe un element al listei are ca efect un eveniment de acțiune.

Puteți suprascrie metoda `action()` pentru a trata elementul din listă pe care s-a efectuat dublu-clic. Pentru selecțiile și deselectiile din listă, puteți să suprascrieți metoda `handleEvent()` și să testați identificatorii de eveniment `LIST_SELECT` și `LIST_DESELECT`.

Evenimente ale barelor de derulare

Numai pentru diferitele tipuri de mișcări ale acestora este generată o întreagă gamă de evenimente. Pentru toate acestea va trebui să folosiți metoda `handleEvent()`.

<u>Identificatorul evenimentului</u>	<u>Ce reprezintă</u>
SCROLL_ABSOLUTE	Generat la deplasarea casetei barei de derulare
SCROLL_LINE_DOWN	Generat la selectarea butonului din capătul de jos sau din stânga al barei de derulare
SCROLL_LINE_UP	Generat la selectarea butonului din capătul de sus sau din dreapta al barei de derulare
SCROLL_PAGE_DOWN	Generat la selectarea spațiului aflat dedesubtul (sau la stânga) casetei barei de derulare
SCROLL_PAGE_UP	Generat la selectarea spațiului aflat deasupra (sau la dreapta) casetei barei de derulare

Ferestre, cadre și casete de dialog. Clasele Window

Clasele AWT care produc ferestre sau casete de dialog moștenesc o clasă comună, **Window**. Clasa **Window** moștenește clasa **Container**, ca și componentele panou și applet, și oferă un comportament generic pentru toate elementele derivate din ferestre.

De obicei nu se folosesc instanțe ale clasei **Window**, ci ale celor două subclase principale ale sale: **Frame** și **Dialog**.

Clasa **Frame** (cadru) oferă o fereastră care conține o bară de titlu, butoane de închidere și alte elemente specifice unei ferestre pentru un anumit sistem. De asemenea, cadrele permit adăugarea unor bare de meniuri.

Clasa **Dialog** este o variantă mai limitată a clasei **Frame**, care, de obicei, nu are un titlu. **FileDialog**, o subclasă a clasei **Dialog**, oferă o casetă de dialog standard pentru alegerea unui fișier (de obicei, aceasta poate fi folosită doar în cadrul aplicațiilor Java, deoarece applet-urile impun restricții de securitate).

Pentru a adăuga o fereastră sau o casetă de dialog în applet-ul sau aplicația dumneavoastră trebuie să creați subclase ale claselor **Frame** sau **Dialog**.

Cadre

Cadrele (frame)

Un cadru reprezintă o fereastră specifică unei anumite platforme, care conține un titlu, o bară de meniuri, butoane de închidere și de redimensionare și alte elemente specifice unei ferestre.

Pentru a crea un cadru se folosește unul din următorii constructori:

new Frame () creează un cadru simplu, fără titlu.

new Frame (String) creează un cadru simplu, cu titlul dat de șirul primit ca argument.s) sunt ferestre independente

Deoarece cadrele moștenesc clasa **Window**, care moștenește clasa **Container**, care moștenește clasa **Component**, cadrele sunt create și folosite cam în același fel cu celelalte componente AWT.

Disponerea prestabilită pentru cadre este **BorderLayout**.

Example

```
win = new Frame("Fereastra mea") ;
```

```
win.setLayout(new BorderLayout(10, 20));
```

```
win.add("North", new Button("Start")) ;
```

```
win.add("Center", new Button("Mutare"));
```

Pentru a stabili dimensiunea noului cadru se folosește metoda **resize ()**:

```
win.resize(100, 200);
```

Metoda **pack()**, care nu primește argumente, creează o fereastră de cea mai mică dimensiune posibilă,

Pentru a afișa o fereastră se folosește metoda **show()**:

```
win.show() ; win.hide();
```

Casete de dialog

Sunt destinate pentru a fi folosite drept ferestre temporare – adică pentru a vă prezenta mesaje de avertizare, pentru a vă cere anumite informații și așa mai departe.

AWT oferă două tipuri de casete de dialog: clasa **Dialog**, care produce o casetă de dialog generică, și **FileDialog**, care produce o casetă de dialog pentru căutarea de fișiere, specifică platformei respective.

Ele pot fi definite așa încât să nu fie redimensionabile sau pot fi definite drept *modale*.

Obiecte dialog

Casetele de dialog se creează și se folosesc asemănător ferestrelor. Pentru crearea unei casete de dialog generice se folosește unul din următorii constructori:

Dialog(Frame, boolean) creează o casetă de dialog invizibilă, atașată cadrului curent, fie modală (true), fie nemodală (false).

Dialog(Frame, String, boolean) creează o casetă de dialog invizibilă, cu titlul dat de argumentul șir, atașată cadrului curent, modală (true) sau nu (false).

Fereastra de dialog, ca și cadrul de fereastră (frame), este un panou pe care se pot dispune și desena componente de interfață sau pe care se pot realiza operații grafice, la fel cum s-ar proceda cu oricare alt panou. Ca și alte ferestre, și cea de dialog este inițial invizibilă, însă o puteți afișa sau ascunde folosind metodele **show()** , respectiv **hide()**.

Casete de dialog pentru sistemul de fișiere

Clasa **FileDialog** permite crearea unei casete de dialog de tip File Open/Save (pentru deschiderea sau salvarea fișierelor), care vă permite să accesați sistemul local de fișiere.

Pentru a crea o casetă de dialog pentru fișiere, folosiți unul din următorii constructori:

FileDialog(Frame, String) creează o casetă de dialog atașată unui cadru dat și cu un titlu dat, pentru încărcarea unui fișier.

FileDialog(Frame, String, int) creează tot o casetă de dialog, însă argumentul întreg este folosit pentru a stabili dacă este vorba despre o casetă de dialog pentru încărcarea sau pentru salvarea unui fișier. (Singura diferență este eticheta butoanelor; de fapt, această casetă de dialog nu deschide sau nu salvează nimic.) Opțiunile pentru acest argument sunt **FileDialog.LOAD** sau **FileDialog.SAVE**.

Evenimente de fereastră

Evenimentele de fereastră apar atunci când starea unei ferestre se modifică: de exemplu, când este mutată, redimensionată, minimizată, restabilită, mutată în prim-plan sau închisă.

metoda **handleEvent()**

Numele evenimentului	Când apare
WINDOW_DESTROY	Generat atunci când o fereastră este închisă folosind butonul Close sau opțiunea de meniu Close
WINDOW_EXPOSE	Generat atunci când fereastra este adusă în prim-plan din spatele altor ferestre
WINDOW_ICONIFY	Generat atunci când fereastra este minimizată (redușă la dimensiunea unei pictograme)
WINDOW_DEICONIFY	Generat atunci când fereastra este restabilită (când se revine de la starea de pictogramă)
WINDOW_MOVED	Generat atunci când fereastra este mutată

Meniuri

O bară de meniuri (menu bar) este o colecție de meniuri. Un meniu, în schimb, conține o colecție de opțiuni, care pot avea denumiri și, opțional, prescurtări (shortcuts). Biblioteca AWT oferă clase pentru toate aceste elemente de meniu, cum ar fi **MenuBar**, **Menu** și **MenuItem**.

Bare de meniuri și meniuri

```
MenuBar mbar = new Menubar();  
fereastră.setMenuBar(mbar);  
Menu meniulMeu = new Menu("File") ;  
mbar.add(meniulMeu) ;
```

Elemente de meniu

instanțe ale clasei **MenuItem**, pentru elemente de meniu obișnuite

Instanțe ale clasei **CheckboxMenuItem**, pentru elemente de meniu care au stările activat/dezactivat

Alte meniuri, care conțin propriile elemente de meniu

Separatori, pentru liniile care separă grupuri de elemente de meniu

Crearea elementelor de meniu

Elementele (opțiunile) de meniu obișnuite sunt create și adăugate într-un meniu folosind clasa **MenuItem**. Mai întâi se creează o nouă instanță a clasei **MenuItem**, după care se adaugă o componentă **Menu** folosind metoda **add()**:

```
Menu meniulMeu = new Menu("Utilitare");
```

```
MenuItem m1= new MenuItem("Info")
```

```
meniulMeu.add(m1) ;
```

```
meniulMeu.add(new MenuItem("Culori")) ;
```

Submeniurile pot fi adăugate prin simpla creare a unei noi instanțe a clasei **Menu** și adăugarea în primul meniu. După aceasta, puteți adăuga elemente în acest meniu:

```
Menu submeniu = new Menu("Dimensiuni") ;
```

```
meniulMeu.add(submeniu) ;
```

```
submeniu.add(new MenuItem("Mic")) ;
```

```
submeniu.add(new MenuItem("Mediu")) ;
```

```
submeniu.add(new MenuItem("Mare"));
```


Crearea unor aplicații AWT independente

Pentru crearea unei aplicații Java *clasa aplicației principale trebuie să moștenească clasa Frame*.

```
import java.awt.* ;
```

```
class AplicatieAWT extends Frame {
    AplicatieAWT(String titlu) {
        super(titlu) ;
        setLayout(new FlowLayout()) ;
        add(new Button("OK")) ;
        add (new Button("Reset"));
        add( new Button("Cancel")) ;
    }
    public static void main(String args[ ]) {
        AplicatieAWT apl = new AplicatieAWT("Prima aplicatie") ;
        apl.resize(300, 300);
        apl.show() ;
    }
    public void închidereFereastră(WindowEvent e) {
        fr.hide() ;
        fr.destroy() ;
        System.exit(0) ;
    }
}
```